UNIVERSIDAD AUTÓNOMA DEL ESTADO DE HIDALGO

Instituto de Ciencias Básicas e Ingeniería

Licenciatura en Ciencias Computacionales

Práctica 3

Construcción de AFD

Alumna: Pilar Bucio Hidalgo

Catedrático: EDUARDO CORNEJO VELÁZQUEZ

Grupo: 6-3

Asignatura: Autómatas y Compiladores

Práctica 3: Simulador de Autómata Finito Determinista (AFD)

Los Autómatas Finitos son máquinas teóricas que van cambiando de estado dependiendo de la entrada que reciben. La salida se limita a dos valores: aceptado y no aceptado, para indicar si la cadena que se ha recibido como entrada es o no válida.

Un Autómata Finito Determinista (AFD) se define como una tupla:

$$AFD = (\Sigma, Q, \delta, q_0, F)$$

- Σ es el alfabeto de entrada.
- Q es el conjunto finito y no vac´ıo de los estados.
- δ es la función de transición, $\delta : Q \times \Sigma \rightarrow Q$.
- $q_0 \in Q$ es el estado inicial.
- $F \subset Q$ es un conjunto de estados finales de aceptación ($F = \emptyset$).

Sea $A = (\Sigma, Q, \delta, q_0, F)$ un AFD y sea $w = w_0 w_1 \ldots w_{n-1}$ una cadena de s´ımbolos donde $w_i \in \Sigma$. Entonces, A acepta w si existe una secuencia de estados $r_0, r_1, \ldots, r_n \in Q$ con tres condiciones:

1. $r_0 = q_0$
2. $\delta(r_i, w_i) = r_{i+1}$ para $i = 0, 1, \ldots, n - 1$
3. $r_n \in F$

La condición 1 establece que la máquina comienza en el estado inicial.
La condición 2 establece que la máquina cambia desde un estado hacia otro estado de acuerdo a la función de transición.
La condición 3 establece que la máquina acepta la cadena de entrada si termina en un estado de aceptación.

Entonces, A reconoce el lenguaje L si $L = \{w \mid A \text{ acepta } w\}$.

3. OBJETIVO GENERAL

Construir un programa en un lenguaje de programación de alto nivel para implementar un Autómata Finito Determinista (AFD) que permita analizar un conjunto de cadenas de s´ımbolos (W) para determinar si pertenecen o no al lenguaje (L) reconocido por el AFD.

4. OBJETIVOS ESPECÍFICOS

- Identificar los componentes de un Autómata Finito Determinista (AFD).
- Implementar un AFD en un lenguaje de programación de alto nivel.
- Determinar si diferentes conjuntos de s´ımbolos (W) pertenecen al lenguaje (L) reconocido por el AFD.

5. EQUIPO Y SOFTWARE A UTILIZAR

Equipo de cómputo con software para programación C++ y compilador GCC/G++.

6. PROCEDIMIENTO PARA EL DESARROLLO DE LA PRÁCTICA

Se construyó una solución en el lenguaje C++ que procesa la entrada en el formato requerido por la gu´ıa de la práctica, leyendo parámetros iniciales (N, S, D, q_0, T, C), alfabeto Σ, estados de aceptación F, D transiciones y C cadenas W.

Código Fuente del Programa (C++)

```cpp
#include <iostream>
#include <string>
#include <vector>
#include <set>
#include <map>
#include <sstream>

using namespace std;

void simular_afd() {
    int N, S, D, q0, T, C;
    if (!(cin >> N >> S >> D >> q0 >> T >> C)) {
        return;
    }

    set<string> alfabeto;
    for (int i = 0; i < S; ++i) {
        string sim;
        cin >> sim;
        alfabeto.insert(sim);
    }

    set<int> estados_finales;
    for (int i = 0; i < T; ++i) {
        int est;
        cin >> est;
        estados_finales.insert(est);
    }

    map<pair<int, string>, int> transiciones;
    for (int i = 0; i < D; ++i) {
        int origen, destino;
        string simbolo;
        cin >> origen >> simbolo >> destino;
        transiciones[{origen, simbolo}] = destino;
    }

    string dummy;
    getline(cin, dummy);

    for (int i = 0; i < C; ++i) {
        string cadena;
        getline(cin, cadena);

        if (!cadena.empty() && cadena.back() == '\r') {
            cadena.pop_back();
        }

        int estado_actual = q0;
```

```
        bool cadena_valida = true;

        for (char c : cadena) {
            string simbolo(1, c);
            if (alfabeto.count(simbolo) && transiciones.count({
    estado_actual, simbolo})) {
                estado_actual = transiciones[{estado_actual,
    simbolo}];
            } else {
                cadena_valida = false;
                break;
            }
        }

        if (cadena_valida && estados_finales.count(estado_actual)
    ) {
            cout << "ACEPTADA" << "\n";
        } else {
            cout << "RECHAZADA" << "\n";
        }
    }
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    simular_afd();
    return 0;
}
```

Evidencia de Ejecución

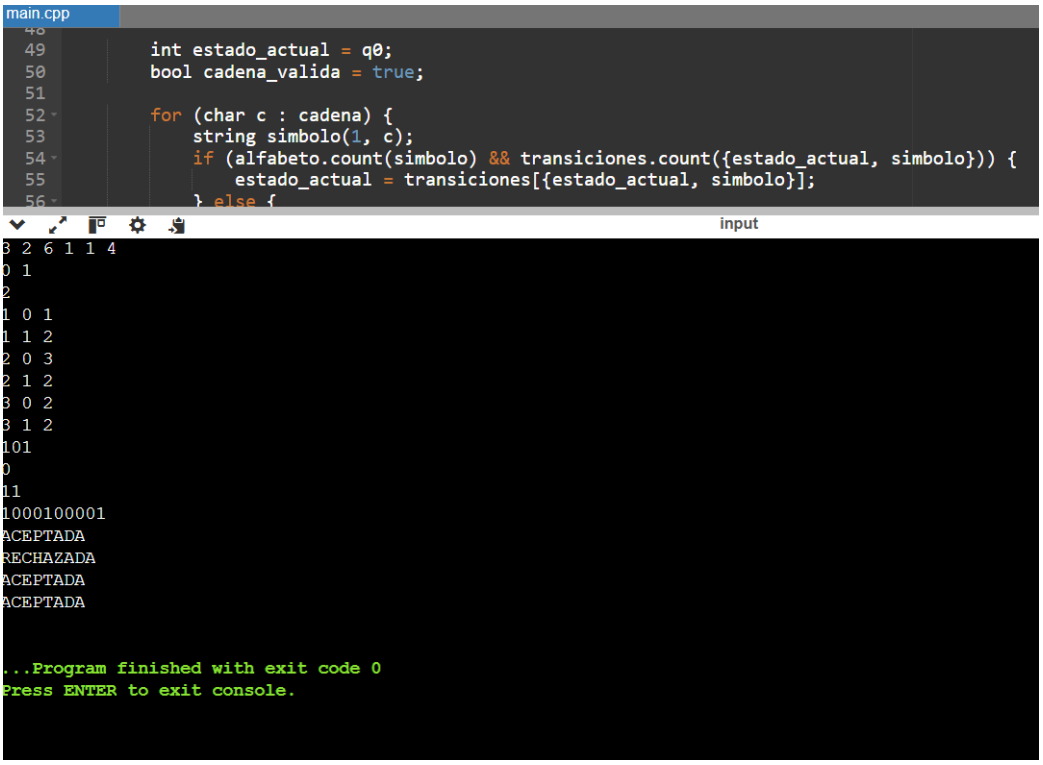


Figura 1: Captura de pantalla con la ejecución del programa C++ y validación del juego de pruebas.

7. CUESTIONARIO

1. **¿Qué paradigma de programación utilizaste para la implementación del AFD?, ¿Cuáles elementos del lenguaje de programación utilizaste?**

   Se utilizó el paradigma **imperativo / estructurado**. Se emplearon elementos nativos de C++ como flujo de entrada y salida estándar (std::cin, std::cout), estructuras de control (for, if-else, break), tipos de datos primitivos (int, char), manejo de cadenas (std::string) e inclusión de librerías del STL.

2. **¿Qué estructuras de datos utilizaste para implementar los componentes del AFD?**

   - std::map<std::pair<int, std::string>, int>: Representa la función de transición δ, mapeando cada par (estado_origen, simbolo) hacia su estado_destino.
   - std::set<std::string> y std::set<int>: Utilizados para representar el alfabeto Σ y el conjunto de estados finales F, respectivamente, garantizando búsquedas en tiempo logarítmico.
   - std::string: Para almacenar y recorrer secuencialmente las palabras W.

3. **¿Qué elementos del lenguaje de programación te permitieron implementar las funciones de transición?, ¿cuáles son las limitaciones del programa que construiste?**

   Se utilizó la estructura std::map en combinación con parejas ordenadas std::pair, relacionando directamente cada tupla ($Q \times \Sigma$) con su estado destino.

   Limitaciones:

   - Solo simula Autómatas Finitos **Deterministas** (AFD). No soporta no-determinismo (AFND) ni transiciones nulas (ϵ).
   - La entrada requiere apegarse strictly al formato delimitado por espacios y saltos de línea sin caracteres de control no contemplados.

4. **¿Qué condiciones se deben cumplir para que el programa pueda evaluar cualquier AFD?**

   - Que el autómata definido cumpla formalmente la propiedad de determinismo (un único destino por par estado-símbolo).
   - Que los identificadores numéricos de los estados estén definidos en el rango $1 \leq q \leq N$.
   - Que los datos de entrada mantengan el orden exacto de parámetros e instrucciones requerido por la especificación de la práctica.

8. CONCLUSIONES

Durante el desarrollo de esta práctica se logró implementar exitosamente un simulador de Autómata Finito Determinista (AFD) utilizando el lenguaje C++. Se comprendió la representación formal del autómata $AFD = (\Sigma, Q, \delta, q_0, F)$ y su traducción a estructuras de datos eficientes.

El uso de mapeos (std::map) para implementar la función de transición δ permitió realizar consultas en tiempo constante, mientras que los conjuntos (std::set) facilitaron la validación rápida de pertenencia de símbolos y estados finales. Con ello, se comprobó experimentalmente la capacidad del programa para procesar y clasificar múltiples cadenas de texto, determinando de forma precisa si pertenecen o no al lenguaje reconocido por el autómata.

9. BIBLIOGRAFÍA

- Giró, J., Vázquez, J., Meloni, B., Constable, L. (2015). *Lenguajes formales y teoría de autómatas*. Editorial Alfaomega. Argentina.
- Ruiz Catalán, J. (2010). *Compiladores: teoría e implementación*. Editorial Alfaomega. México.
- Brookshear, J. G. (1995). *Teoría de la Computación. Lenguajes formales, autómatas y complejidad*. Editorial Addison-Wesley Iberoamericana. Primera edición. U.S.A.